

Fișă de lucru – Clasa list

Realizată de: Aloman Denisa-Maria

Teorie

1.1 Ce este o listă?

O **listă** este o colecție ordonată de elemente, care poate conține valori de orice tip (numere întregi, șiruri de caractere, valori reale etc.). O listă este **mutabilă**: elementele sale pot fi modificate după creare.

```
note = [8, 10, 7, 9, 6]           # lista de numere intregi
nume = ["Ana", "Ion", "Maria"]   # lista de siruri de caractere
mixt = [1, "doi", 3.0, True]     # lista cu tipuri mixte
vida = []                        # lista vida
```

1.2 Indexare și acces la elemente

Fiecare element dintr-o listă are asociat un index - un număr întreg care indică poziția sa. Indexarea începe de la 0. Se pot folosi și **indecși negativi** (-1 = ultimul element).

8	10	7	9	6
0	1	2	3	4
-5	-4	-3	-2	-1

```
note = [8, 10, 7, 9, 6]
print(note[0])    # 8    (primul element)
print(note[-1])   # 6    (ultimul element)
print(note[2])    # 7
```

1.3 Modificarea elementelor

Elementele unei liste **pot fi modificate** direct prin index:

```
note = [8, 10, 7, 9, 6]
note[2] = 10      # inlocuim 7 cu 10
print(note)       # [8, 10, 10, 9, 6]
```

1.4 Funcții built-in utile

Funcția	Ce face	Exemplu
<code>len(lst)</code>	Returnează numărul de elemente	<code>len([1,2,3]) → 3</code>
<code>min(lst)</code>	Returnează valoarea minimă	<code>min([8,10,7]) → 7</code>
<code>max(lst)</code>	Returnează valoarea maximă	<code>max([8,10,7]) → 10</code>
<code>sum(lst)</code>	Calculează suma elementelor	<code>sum([8,10,7]) → 25</code>

1.5 Metode uzuale ale clasei `list`

Metoda	Ce face	Exemplu
<code>append(x)</code>	Adaugă <code>x</code> la sfârșitul listei	<code>lst.append(5)</code>
<code>insert(i, x)</code>	Inserează <code>x</code> la poziția <code>i</code>	<code>lst.insert(0, 99)</code>
<code>pop(i)</code>	Elimină și returnează elementul de la poziția <code>i</code> (implicit: ultimul)	<code>lst.pop()</code>
<code>remove(x)</code>	Elimină prima apariție a valorii <code>x</code>	<code>lst.remove(7)</code>
<code>sort()</code>	Sortează lista în ordine crescătoare	<code>lst.sort()</code>
<code>reverse()</code>	Inversează ordinea elementelor	<code>lst.reverse()</code>
<code>count(x)</code>	Numără aparițiile valorii <code>x</code>	<code>lst.count(8) → 2</code>
<code>index(x)</code>	Returnează indexul primei apariții a <code>x</code>	<code>lst.index(10) → 1</code>
<code>copy()</code>	Returnează o copie a listei	<code>lst2 = lst.copy()</code>
<code>clear()</code>	Șterge toate elementele	<code>lst.clear()</code>

1.6 Operatori specifici

```
a = [1, 2, 3]
b = [4, 5]

print(a + b)      # [1, 2, 3, 4, 5]    (concatenare)
print(a * 2)      # [1, 2, 3, 1, 2, 3] (repetare)
print(3 in a)     # True               (apartenenta)
print(6 not in a) # True               (non-apartenenta)
print(a == [1,2,3]) # True            (egalitate)
```

1.7 Parcurgerea unei liste

În Python există două moduri de a parcurge o listă, fiecare potrivit pentru situații diferite:

- **Varianta 1 - directă (element cu element):** folosim `for element in lista` când ne interesează doar *valoarea* fiecărui element, nu poziția lui.
- **Varianta 2 - prin index:** folosim `for i in range(len(lista))` când avem nevoie și de *poziția* elementului (de exemplu, pentru a-l modifica sau pentru a-l compara cu vecinii săi).

```
note = [8, 10, 7, 9, 6]

# Varianta 1: direct, element cu element
for nota in note:
    print(nota)

# Varianta 2: prin index
for i in range(len(note)):
    print(i, note[i])
```

Afișare:

```
# Varianta 1:
8
10
7
9
6

# Varianta 2:
0 8
1 10
2 7
3 9
4 6
```

1.8 Citirea unei liste de la tastatură

```
n = int(input("Numarul de elemente: "))
lst = []
for i in range(n):
    x = int(input("Elementul " + str(i) + ": "))
    lst.append(x)
print(lst)
```

Exemple rezolvate

Exemplul 1 – Calculează media notelor

Cerință: Citește n note numere întregi de la tastatură și afișează media lor.

Exemplu de rulare:

```
Input: n=4, note: 8 10 7 9
Output: Media: 8.5
```

Caz particular:

```
Input: n=0, note:
Output: Numarul introdus nu este valid!
```

Date de intrare: numărul n și cele n note.

Date de ieșire: media aritmetică a notelor.

Restricții: $n > 0$

Pași de rezolvare:

1. Citim un număr natural n , reprezentând numărul de note.
2. Verificăm dacă numărul introdus respectă restricția $n > 0$.
3. Inițializăm o listă vidă: **note**, în care vom reține notele citite.
4. Folosind o instrucțiune repetitivă, vom introduce de la tastatură n note, pe care le adăugăm una câte una la sfârșitul listei **note**.
5. Calculăm media împărțind suma tuturor notelor din listă la numărul lor.
6. Afișăm media calculată.

Rezolvare:

```
n = int(input("Numarul de note: ")) # pas 1: citim un nr intreg

if n <= 0:                          # pas 2: verificarea restrictiei
    print("Numarul introdus nu este valid!")
else:
    note = []                       # pas 3: initializarea listei de note
    for i in range(n):              # pas 4: citirea a n note
        x = int(input("Nota: "))
        note.append(x)             # adaugarea notei in lista
    media = sum(note) / len(note)   # pas 5: calcularea mediei
    print("Media:", media)         # pas 6 : afisarea mediei
```

Exemplul 2 – Elimină duplicatele dintr-o listă

Cerință: Citește n numere întregi de la tastatură și afișează lista fără duplicate, păstrând ordinea primei apariții.

Exemplu de rulare:

```
Input: n=6, elemente: 3 1 2 3 1 5
Output: [3, 1, 2, 5]
```

Caz particular:

```
Input: n=3, elemente: 7 7 7
Output: [7]
# Toate elementele sunt identice -> ramane doar unul.
```

Date de intrare: numărul întreg n și cele n numere întregi.

Date de ieșire: o listă de numere întregi cu elemente distincte, în ordinea primei apariții.

Restricții: $n > 0$

Pași de rezolvare:

1. Citim un număr natural n , reprezentând numărul de elemente.
2. Verificăm dacă n respectă restricția $n > 0$.
3. Inițializăm o listă vidă **rezultat**, în care vom reține elementele distincte ale listei.
4. Folosind o instrucțiune repetitivă, citim pe rând cele n numere întregi de la tastatură. Pentru fiecare număr x citit:
 - (a) Dacă x **nu se află deja** în lista **rezultat**, îl adăugăm la sfârșitul ei.
 - (b) Dacă x **există deja** în lista **rezultat**, îl ignorăm și trecem la următorul element.
5. Afișăm lista **rezultat**, care conține elementele distincte în ordinea primei lor apariții.

Rezolvare:

```
n = int(input("Numarul de elemente: ")) # pas 1: citim numarul
if n <= 0:                               # pas 2: verificarea restrictiei
    print("Numarul introdus nu este valid!")
else:
    rezultat = []                        # pas 3: initializam lista rezultat
    for i in range(n):                  # pas 4: citirea celor n elemente
        x = int(input("Element: "))
        if x not in rezultat:          # verificam daca x e deja in
            # lista
            rezultat.append(x)          # adaugam doar daca nu exista
    print(rezultat)                     # pas 5: afisam lista finala
```

Exemplul 3 – Inversează o listă fără reverse()

Cerință: Citește n numere întregi de la tastatură și afișează lista în ordine inversă, fără a folosi metoda `reverse()` sau slicing.

Exemplu de rulare:

```
Input: n=4, elemente: 1 2 3 4
Output: [4, 3, 2, 1]
```

Caz particular:

```
Input: n=1, element: 5
Output: [5]
# O lista cu un singur element este identica cu inversul ei.
```

Date de intrare: numărul natural n și cele n numere întregi.

Date de ieșire: o listă cu aceleași elemente, așezate în ordine inversă.

Restricții: $n > 0$

Pași de rezolvare:

1. Citim un număr natural n , reprezentând numărul de elemente.
2. Verificăm dacă n respectă restricția $n > 0$.
3. Inițializăm o listă vidă `lst`, în care vom reține elementele citite de la tastatură.
4. Folosind o instrucțiune repetitivă, citim pe rând cele n numere întregi și le adăugăm unul câte unul la sfârșitul listei `lst`.
5. Inițializăm o a doua listă vidă `inversat`, în care vom construi lista în ordine inversă.
6. Parcurgem lista `lst` de la ultimul element spre primul și adăugăm fiecare element întâlnit la sfârșitul listei `inversat`.
7. Afișăm lista `inversat`.

Rezolvare:

```
n = int(input("Numarul de elemente: ")) # pas 1: citim numarul
if n <= 0:                               # pas 2: verificarea restrictiei
    print("Numarul introdus nu este valid!")
else:
    lst = []                             # pas 3: initializam lista
    for i in range(n):                   # pas 4: citirea celor n elemente
        x = int(input("Element: "))
        lst.append(x)
    inversat = []                         # pas 5: initializam lista inversat
    for i in range(n - 1, -1, -1):       # pas 6: parcurgere invers
        inversat.append(lst[i])
    print(inversat)                       # pas 7: afisam lista inversata
```

Probleme

1. Ce se afișează? (tracing de cod)

Se consideră următorul program Python:

```
lst = [3, 1, 4, 1, 5, 9, 2, 6]
rezultat = []
for x in lst:
    if x % 2 == 0:
        rezultat.append(x)
print(rezultat)
```

Cerință: Scrie ce se afișează și explică ce face programul.

Indicații:

- Ce condiție trebuie să îndeplinească un element pentru a fi adăugat?
- Parcurge lista element cu element și urmărește când condiția este adevărată.

Răspuns:

2. Ce se afișează? (metode ale listei)

Se consideră următorul program Python:

```
lst = [5, 3, 8, 1, 7]
lst.sort()
lst.append(10)
lst.remove(3)
print(lst)
print(lst.index(8))
```

Cerință: Scrie ce se afișează după fiecare modificare.

Indicații:

- Urmărește starea listei după fiecare operație.
- Ce face `index()` dacă elementul există în listă?

Răspuns:

3. Citește `n` numere întregi și afișează:

- cel mai mare element din listă;
- de câte ori apare cel mai mare element;
- indexul primei apariții a celui mai mare element.

Exemplu:

```
Input: n=5, elemente: 3 7 2 7 4
Output:
Maximul: 7
Apare de: 2 ori
Prima aparitie la indexul: 1
```

Indicații:

- Există funcții și metode care rezolvă direct fiecare subpunct.
- Poți rezolva toate cele trei cerințe fără nicio buclă `for`.

Rezolvare:

4. Citește **n** numere întregi și construiește două liste: una cu elementele de pe pozițiile pare și una cu elementele de pe pozițiile impare. Afișează ambele liste.

Exemplu:

```
Input: n=6, elemente: 10 20 30 40 50 60
Output:
Pozitii pare:    [10, 30, 50]
Pozitii impare: [20, 40, 60]
```

Indicații:

- Cum determini dacă un index este par sau impar?
- Ai nevoie de index la parcurgere, nu doar de valoare.

Rezolvare:

5. Citește **n** numere întregi și mută toate elementele egale cu 0 la sfârșitul listei, păstrând ordinea relativă a celorlalte elemente.

Exemplu:

```
Input: n=6, elemente: 1 0 3 0 5 2
Output: [1, 3, 5, 2, 0, 0]
```

Indicații:

- Construiește mai întâi lista fără zerouri.
- Câte zerouri ai eliminat? Adaugă-le la final.

Rezolvare:

6. Citește două liste de câte **n** numere întregi și construiește o a treia listă care conține doar elementele comune celor două liste (fără duplicate). Afișează lista rezultată.

Exemplu:

```
Input: lst1 = [1, 2, 3, 4, 5], lst2 = [3, 4, 5, 6, 7]  
Output: [3, 4, 5]
```

Indicații:

- Parcurge prima listă și verifică dacă fiecare element se află și în a doua.
- Cum eviți să adaugi un element de două ori în rezultat?

Rezolvare: